

Web Components I

10 ejercicios prácticos — custom elements, plantillas HTML y Shadow DOM

DOCUMENTO DE REFERENCIA

Frontend 01

Estos ejercicios repasan qué son los web components, cómo se definen custom elements, cómo se usan las plantillas HTML (`<template>` y `<slot>`) y cómo funciona el Shadow DOM.

Fácil (1-3)

Fácil-Medio (4-7)

Medio (8-10)

1 Qué es un web component

FÁCIL

Explica con tus palabras qué es un web component y escribe un ejemplo de nombre válido para un custom element (recuerda: debe contener un guion).

2 `customElements.define()`

FÁCIL

Explica para qué sirve el método `customElements.define()` y qué dos argumentos necesita como mínimo.

3 `template` vs `slot`

FÁCIL

Explica la diferencia entre la etiqueta `<template>` y la etiqueta `<slot>`, y en qué se diferencia su comportamiento del resto de HTML.

4 Crear un custom element básico

FÁCIL-MEDIO

Crea un web component `<my-card>` que, usando `customElements.define` y Shadow DOM, muestre un `<h3>Tarjeta</h3>` dentro de un contenedor con un borde de 1px. Escribe el código JavaScript completo del constructor.

5 Añadir un slot con contenido por defecto

FÁCIL-MEDIO

Amplía el componente `<my-card>` del ejercicio anterior añadiendo un `<slot name="contenido">` con el texto por defecto "Sin contenido", de forma que se pueda personalizar al usar el componente: `<my-card><span slot="contenido">Hola</span></my-card>`.

6 Ciclo de vida del custom element

MEDIO

Indica qué callback del ciclo de vida se ejecuta en cada uno de estos casos: (a) el elemento se añade al DOM por primera vez; (b) el elemento se elimina del DOM; (c) uno de sus atributos observados cambia de valor.

7 Shadow DOM open vs closed

MEDIO

Explica la diferencia entre crear un Shadow DOM con `{ mode: 'open' }` y con `{ mode: 'closed' }`, y qué consecuencia práctica tiene elegir cada uno a la hora de inspeccionar el componente desde fuera.

8 Encapsulación de estilos

MEDIO

Un desarrollador define en su hoja de estilos global `button { background: red; }` pero el botón dentro de su web component sigue viéndose gris. Explica por qué ocurre esto, basándote en el concepto de Shadow DOM.

9 Web components y frameworks: verdadero o falso

MEDIO

Indica si son verdaderas o falsas las siguientes afirmaciones, justificando la respuesta: (1) React, Angular y Vue crean web components completamente distintos, sin relación con los nativos. (2) Los frameworks añaden una capa que simplifica la creación de web components y les da funcionalidades extra por defecto.

10 Ejercicio integrador:

MEDIO

Diseña un componente reutilizable `<my-alert type="success"></my-alert>` que muestre un mensaje (a través de un slot) con un color de fondo distinto según el atributo `type` ("success" en verde, "error" en rojo). Describe el HTML interno, los estilos y el JavaScript necesario, incluyendo cómo leerías el atributo `type` desde el componente.

Soluciones orientativas

1. Un web component es un elemento HTML personalizado y reutilizable. Ejemplo válido: `<user-card>` (todo nombre de custom element debe llevar al menos un guion).
2. Sirve para registrar un nuevo elemento personalizado en el navegador. Necesita como mínimo el nombre del elemento (string con guion) y la clase que extiende de `HTMLElement`.
3. `<template>` define estructuras HTML reutilizables que no se renderizan por sí mismas hasta que se clonan por JavaScript; `<slot>` define un hueco dentro del componente que se puede rellenar con contenido externo al usarlo.
4.

```
class MyCard extends HTMLElement { constructor() { super(); this.attachShadow({mode:'open'}); this.shadowRoot.innerHTML = '<style>div{border:1px solid #ccc;}</style><div><h3>Tarjeta</h3></div>'; } } customElements.define('my-card', MyCard);
```
5. Se añade `<slot name="contenido">Sin contenido</slot>` dentro del shadowRoot; al usar el componente con un `<span slot="contenido">`, ese contenido sustituye al texto por defecto.
6. (a) `connectedCallback`; (b) `disconnectedCallback`; (c) `attributeChangedCallback`.
7. Con `open` se puede acceder al Shadow DOM desde fuera mediante `elemento.shadowRoot`; con `closed` esa propiedad devuelve `null` y el árbol interno queda oculto para el código externo.
8. El botón está dentro del Shadow DOM del componente, que aísla sus estilos del resto de la página: los estilos globales no entran al Shadow DOM y los internos no salen de él, salvo que se expongan explícitamente mediante atributos o variables CSS.
9. (1) Falsa: los frameworks se apoyan en la misma base nativa (custom elements, Shadow DOM, templates). (2) Verdadera: los frameworks simplifican su creación y añaden funcionalidades por defecto encima de la base nativa.
10. El componente define en su Shadow DOM un contenedor con un `<slot>` para el mensaje; en `connectedCallback` o mediante `attributeChangedCallback` (observando `type`) se lee `this.getAttribute('type')` y se aplica una clase o estilo distinto (verde/rojo) al contenedor según su valor.

Practica primero sin mirar las soluciones — es la mejor forma de consolidar lo aprendido.